

1. Elija un sistema real de su área de interés

Sistema elegido

Sistema de optimización de campañas de marketing digital, donde una IA analiza el rendimiento de las campañas y decide cómo distribuir el presupuesto de manera automática.

(a) Estado S_t

El estado representa toda la información disponible para tomar una decisión en un instante determinado, por ejemplo:

- Presupuesto disponible.
- CTR (Click Through Rate).
- CPC (Cost Per Click).
- Número de conversiones.
- ROAS (Return on Ad Spend).
- Rendimiento histórico de cada campaña.

Mientras más completo sea el estado, mejores decisiones podrá tomar el agente.

(b) Espacio de acciones A

Las acciones posibles incluyen:

- Aumentar el presupuesto de una campaña.
- Reducir el presupuesto.
- Pausar una campaña.
- Reactivar una campaña.
- Cambiar la estrategia de puja.
- Redistribuir presupuesto entre campañas.

(c) Función de recompensa R

La recompensa debe reflejar los objetivos del negocio.

Se puede:

- Recompensar cuando aumentan las conversiones.
- Recompensar cuando mejora el ROAS.
- Penalizar cuando se consume mucho presupuesto sin obtener buenos resultados.

El sistema aprende a optimizar la inversión y no únicamente a conseguir más clics.

(d) Observabilidad del entorno

El entorno es parcialmente observable.

Aunque el sistema dispone de gran cantidad de datos (CTR, CPC, conversiones, etc.), nunca conoce toda la información relevante. Por ejemplo:

- No sabe exactamente qué piensa cada usuario.
- No conoce si una persona comprará más adelante.
- No puede observar la influencia de anuncios vistos en otras plataformas.

2. Valor apropiado de γ

Para este sistema usaríamos:

$$\gamma = 0.95$$

porque en marketing digital muchas decisiones producen resultados después de cierto tiempo. Una campaña nueva puede requerir varios días para encontrar la audiencia adecuada antes de generar conversiones rentables.

Ejemplo

- Opción A: recompensa inmediata de +30.
- Opción B: pérdida inicial de -10, seguida de una recompensa futura de +100.

Con $\gamma = 0.95$

$$-10 + 0.95(100) = 85$$

El agente considera que vale la pena asumir la pérdida inicial porque la recompensa futura es mucho mayor.

Con $\gamma = 0.01$

$$-10 + 0.01(100) = -9$$

En este caso el agente prácticamente ignora la recompensa futura y evita invertir en campañas nuevas, aunque sean mucho más rentables a largo plazo.

3. Política determinista vs. política estocástica

Política determinista

Una política determinista siempre toma la misma decisión para un mismo estado.

Ejemplo:

- Si el ROAS > 4, aumentar el presupuesto un 20%.
- Si el ROAS < 2, reducir el presupuesto.

Ante las mismas condiciones, la decisión siempre será la misma.

Política estocástica

Una política estocástica asigna probabilidades a las acciones posibles.

Por ejemplo:

- 60% de probabilidad de aumentar el presupuesto.
- 25% de mantenerlo igual.
- 15% de probar una nueva audiencia.

No siempre ejecuta la misma acción, incluso cuando el estado es idéntico.

¿Cuándo preferir una política estocástica?

Preferiría una política estocástica cuando el objetivo es explorar nuevas estrategias.

Si el sistema siempre toma la misma decisión, puede quedarse explotando únicamente las campañas que ya funcionan y perder oportunidades de descubrir configuraciones o audiencias que generen un mejor rendimiento en el largo plazo.

Programa

Código

```
"""
Entorno: GridWorld 4x4 simple
- El agente parte de (0,0) y busca llegar a (3,3)
- Recompensa: -1 por cada paso, +10 al llegar a la meta, -5 si cae en celda trampa
- El agente sigue una política aleatoria (no aprende todavía - solo observamos el ciclo)
"""
```

```

import numpy as np
import random

# Constantes
GRID_SIZE = 4
GOAL_STATE = (3, 3) # Estado terminal con recompensa positiva
TRAP_STATES = {(1, 1), (2, 3)} # Estados trampa
MAX_STEPS = 50 # Horizonte máximo del episodio
GAMMA = 0.99 # Factor de descuento

# Acciones
# Espacio de acciones discreto: 4 direcciones posibles
ACTIONS = {
    0: (-1, 0), # Arriba
    1: (1, 0), # Abajo
    2: (0, -1), # Izquierda
    3: (0, 1), # Derecha
}
ACTION_NAMES = {0: "↑", 1: "↓", 2: "←", 3: "→"}

# Entorno
class GridWorldEnv:
    """
    Representa el entorno (Environment) en el vocabulario de RL.

    Responsabilidades:
    - Mantener el estado actual del mundo
    - Transicionar al nuevo estado dado una acción
    - Emitir la recompensa R_{t+1} después de cada transición
    """
    def reset(self):
        """Inicializa el entorno y devuelve el estado inicial S_0."""
        self.state = (0, 0)
        self.done = False
        return self.state

    def step(self, action):
        """
        Ejecuta un paso del ciclo agente-entorno.

        Recibe: A_t (action)
        Devuelve: S_{t+1}, R_{t+1}, done

        Esta función implementa la función de transición P(S_{t+1} | S_t, A_t)
        y la función de recompensa R(S_t, A_t, S_{t+1}).
        """
        if self.done:
            raise RuntimeError("El episodio ya terminó. Llama reset().")

        dr, dc = ACTIONS[action]
        r, c = self.state

        # La transición respeta los bordes del grid (entorno determinista)
        new_r = max(0, min(GRID_SIZE - 1, r + dr))
        new_c = max(0, min(GRID_SIZE - 1, c + dc))
        next_state = (new_r, new_c)

        # Función de recompensa: señal de diseño del ingeniero
        if next_state == GOAL_STATE:
            reward = +10.0 # Meta alcanzada
            self.done = True
        elif next_state in TRAP_STATES:
            reward = -5.0 # Trampa activada
            self.done = True
        else:
            reward = -1.0 # Costo por paso (incentiva llegar rápido)

        self.state = next_state
        return next_state, reward, self.done

# Agente con política aleatoria
class RandomAgent:
    """
    Agente con política estocástica uniforme.

    Política  $\pi(a|s) = 1/|A|$  para todo  $a$  – distribución uniforme.
    No aprende – sirve para ilustrar el ciclo de interacción.
    """
    def __init__(self, action_space_size):
        self.n_actions = action_space_size

    def select_action(self, state):
        """
        Muestra una acción de la política estocástica uniforme.

        En un agente que aprende (Q-learning, PPO, etc.), esta función
        consultaría una tabla Q o una red neuronal.
        """
        return random.randint(0, self.n_actions - 1)

# Ciclo de agente entorno
def run_episode(env, agent, gamma=GAMMA, max_steps=MAX_STEPS, verbose=True):
    """
    Ejecuta un episodio completo y calcula el retorno G_0.

    Implementa directamente el ciclo de la Diapositiva 3:
    S_0 → A_0 → R_1, S_1 → A_1 → R_2, S_2 → ...
    """
    state = env.reset() # S_0
    trajectory = [] # Almacena (S_t, A_t, R_{t+1})
    total_reward = 0.0

    if verbose:
        print(f"\n{'='*50}")

```

```

    print(f"INICIO DEL EPISODIO Estado inicial: {state}")
    print(f"{'-'*50}")

for t in range(max_steps):
    action = agent.select_action(state)          # A_t ~ π(·|S_t)
    next_state, reward, done = env.step(action)  # Transición

    trajectory.append((state, action, reward, next_state))
    total_reward += reward

    if verbose:
        status = "✓ META" if next_state == GOAL_STATE \
            else "x TRAMPA" if next_state in TRAP_STATES \
            else ""
        print(f"t={t:2d} S={state} A={ACTION_NAMES[action]} "
              f"R={reward:+.1f} S'={next_state} {status}")

    state = next_state

    if done:
        break

# Calcula el retorno descontado  $G_0 = \sum \gamma^k * R_{k+1}$ 
# Procesa la trayectoria en reversa para acumulación eficiente
G = 0.0
returns = []
for _, _, r, _ in reversed(trajectory):
    G = r + gamma * G          # G_t = R_{t+1} + γ * G_{t+1}
    returns.insert(0, G)

if verbose:
    print(f"{'-'*50}")
    print(f" Pasos totales : {len(trajectory)}")
    print(f" Recompensa acumulada : {total_reward:+.1f}")
    print(f" Retorno G_0 (γ={gamma}): {returns[0]:.4f}")
    print(f"{'-'*50}\n")

return trajectory, returns

# Experimentar con multiples episodios
def run_experiment(n_episodes=5):
    """
    Corre múltiples episodios para ilustrar la variabilidad de una política
    estocástica – con la misma política, los resultados varían por azar.
    """
    env = GridWorldEnv()
    agent = RandomAgent(action_space_size=len(ACTIONS))

    print("SIMULACIÓN: Ciclo Agente-Entorno en GridWorld 4x4")
    print("Política: aleatoria uniforme (sin aprendizaje)")
    print(f"Factor de descuento γ = {GAMMA}")

    all_returns = []
    for ep in range(n_episodes):
        print(f"\n► Episodio {ep + 1}/{n_episodes}")
        _, returns = run_episode(env, agent, verbose=True)
        all_returns.append(returns[0])

    print("\n" + "="*50)
    print("RESUMEN DEL EXPERIMENTO")
    print(f" Retorno promedio G_0 : {np.mean(all_returns):.4f}")
    print(f" Retorno máximo : {max(all_returns):.4f}")
    print(f" Retorno mínimo : {min(all_returns):.4f}")
    print("="*50)
    print("\nObservación: la variabilidad entre episodios ilustra por qué")
    print("el objetivo de RL es el RETORNO ESPERADO, no un retorno fijo.")

if __name__ == "__main__":
    random.seed(42)
    np.random.seed(42)
    run_experiment(n_episodes=5)

```

Output

SIMULACIÓN: Ciclo Agente-Entorno en GridWorld 4x4
 Política: aleatoria uniforme (sin aprendizaje)
 Factor de descuento γ = 0.99

► Episodio 1/5

```

=====
INICIO DEL EPISODIO Estado inicial: (0, 0)
=====
t= 0 S=(0, 0) A=↑ R=-1.0 S'=(0, 0)
t= 1 S=(0, 0) A=↑ R=-1.0 S'=(0, 0)
t= 2 S=(0, 0) A=← R=-1.0 S'=(0, 0)
t= 3 S=(0, 0) A=↓ R=-1.0 S'=(1, 0)
t= 4 S=(1, 0) A=↓ R=-1.0 S'=(2, 0)
t= 5 S=(2, 0) A=↓ R=-1.0 S'=(3, 0)
t= 6 S=(3, 0) A=↑ R=-1.0 S'=(2, 0)
t= 7 S=(2, 0) A=↑ R=-1.0 S'=(1, 0)
t= 8 S=(1, 0) A=→ R=-5.0 S'=(1, 1) x TRAMPA
=====
Pasos totales : 9
Recompensa acumulada : -13.0
Retorno G_0 (γ=0.99): -12.3393
=====

```

► Episodio 2/5

```

=====
INICIO DEL EPISODIO Estado inicial: (0, 0)
=====
t= 0 S=(0, 0) A=↑ R=-1.0 S'=(0, 0)
t= 1 S=(0, 0) A=↑ R=-1.0 S'=(0, 0)
t= 2 S=(0, 0) A=↑ R=-1.0 S'=(0, 0)
t= 3 S=(0, 0) A=↓ R=-1.0 S'=(1, 0)

```

t= 4	S=(1, 0)	A=↓	R=-1.0	S'=(2, 0)	
t= 5	S=(2, 0)	A=↑	R=-1.0	S'=(1, 0)	
t= 6	S=(1, 0)	A=↓	R=-1.0	S'=(2, 0)	
t= 7	S=(2, 0)	A=→	R=-1.0	S'=(2, 1)	
t= 8	S=(2, 1)	A=↓	R=-1.0	S'=(3, 1)	
t= 9	S=(3, 1)	A=→	R=-1.0	S'=(3, 2)	
t=10	S=(3, 2)	A=←	R=-1.0	S'=(3, 1)	
t=11	S=(3, 1)	A=↑	R=-1.0	S'=(2, 1)	
t=12	S=(2, 1)	A=↓	R=-1.0	S'=(3, 1)	
t=13	S=(3, 1)	A=→	R=-1.0	S'=(3, 2)	
t=14	S=(3, 2)	A=←	R=-1.0	S'=(3, 1)	
t=15	S=(3, 1)	A=←	R=-1.0	S'=(3, 0)	
t=16	S=(3, 0)	A=↓	R=-1.0	S'=(3, 0)	
t=17	S=(3, 0)	A=↓	R=-1.0	S'=(3, 0)	
t=18	S=(3, 0)	A=←	R=-1.0	S'=(3, 0)	
t=19	S=(3, 0)	A=↑	R=-1.0	S'=(2, 0)	
t=20	S=(2, 0)	A=↑	R=-1.0	S'=(1, 0)	
t=21	S=(1, 0)	A=→	R=-5.0	S'=(1, 1)	x TRAMPA

Pasos totales : 22
Recompensa acumulada : -26.0
Retorno G_0 ($\gamma=0.99$): -23.0759

► Episodio 3/5

INICIO DEL EPISODIO Estado inicial: (0, 0)

t= 0	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)	
t= 1	S=(0, 0)	A=←	R=-1.0	S'=(0, 0)	
t= 2	S=(0, 0)	A=←	R=-1.0	S'=(0, 0)	
t= 3	S=(0, 0)	A=←	R=-1.0	S'=(0, 0)	
t= 4	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)	
t= 5	S=(0, 0)	A=→	R=-1.0	S'=(0, 1)	
t= 6	S=(0, 1)	A=↑	R=-1.0	S'=(0, 1)	
t= 7	S=(0, 1)	A=→	R=-1.0	S'=(0, 2)	
t= 8	S=(0, 2)	A=↑	R=-1.0	S'=(0, 2)	
t= 9	S=(0, 2)	A=←	R=-1.0	S'=(0, 1)	
t=10	S=(0, 1)	A=←	R=-1.0	S'=(0, 0)	
t=11	S=(0, 0)	A=↓	R=-1.0	S'=(1, 0)	
t=12	S=(1, 0)	A=↑	R=-1.0	S'=(0, 0)	
t=13	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)	
t=14	S=(0, 0)	A=↓	R=-1.0	S'=(1, 0)	
t=15	S=(1, 0)	A=←	R=-1.0	S'=(1, 0)	
t=16	S=(1, 0)	A=↑	R=-1.0	S'=(0, 0)	
t=17	S=(0, 0)	A=↓	R=-1.0	S'=(1, 0)	
t=18	S=(1, 0)	A=↑	R=-1.0	S'=(0, 0)	
t=19	S=(0, 0)	A=→	R=-1.0	S'=(0, 1)	
t=20	S=(0, 1)	A=←	R=-1.0	S'=(0, 0)	
t=21	S=(0, 0)	A=→	R=-1.0	S'=(0, 1)	
t=22	S=(0, 1)	A=←	R=-1.0	S'=(0, 0)	
t=23	S=(0, 0)	A=↓	R=-1.0	S'=(1, 0)	
t=24	S=(1, 0)	A=←	R=-1.0	S'=(1, 0)	
t=25	S=(1, 0)	A=←	R=-1.0	S'=(1, 0)	
t=26	S=(1, 0)	A=↓	R=-1.0	S'=(2, 0)	
t=27	S=(2, 0)	A=←	R=-1.0	S'=(2, 0)	
t=28	S=(2, 0)	A=↑	R=-1.0	S'=(1, 0)	
t=29	S=(1, 0)	A=↓	R=-1.0	S'=(2, 0)	
t=30	S=(2, 0)	A=↓	R=-1.0	S'=(3, 0)	
t=31	S=(3, 0)	A=↓	R=-1.0	S'=(3, 0)	
t=32	S=(3, 0)	A=→	R=-1.0	S'=(3, 1)	
t=33	S=(3, 1)	A=→	R=-1.0	S'=(3, 2)	
t=34	S=(3, 2)	A=←	R=-1.0	S'=(3, 1)	
t=35	S=(3, 1)	A=↓	R=-1.0	S'=(3, 1)	
t=36	S=(3, 1)	A=←	R=-1.0	S'=(3, 0)	
t=37	S=(3, 0)	A=↑	R=-1.0	S'=(2, 0)	
t=38	S=(2, 0)	A=↓	R=-1.0	S'=(3, 0)	
t=39	S=(3, 0)	A=↑	R=-1.0	S'=(2, 0)	
t=40	S=(2, 0)	A=←	R=-1.0	S'=(2, 0)	
t=41	S=(2, 0)	A=→	R=-1.0	S'=(2, 1)	
t=42	S=(2, 1)	A=←	R=-1.0	S'=(2, 0)	
t=43	S=(2, 0)	A=↑	R=-1.0	S'=(1, 0)	
t=44	S=(1, 0)	A=↓	R=-1.0	S'=(2, 0)	
t=45	S=(2, 0)	A=←	R=-1.0	S'=(2, 0)	
t=46	S=(2, 0)	A=↓	R=-1.0	S'=(3, 0)	
t=47	S=(3, 0)	A=→	R=-1.0	S'=(3, 1)	
t=48	S=(3, 1)	A=→	R=-1.0	S'=(3, 2)	
t=49	S=(3, 2)	A=→	R=+10.0	S'=(3, 3)	✓ META

Pasos totales : 50
Recompensa acumulada : -39.0
Retorno G_0 ($\gamma=0.99$): -32.7771

► Episodio 4/5

INICIO DEL EPISODIO Estado inicial: (0, 0)

t= 0	S=(0, 0)	A=↓	R=-1.0	S'=(1, 0)	
t= 1	S=(1, 0)	A=←	R=-1.0	S'=(1, 0)	
t= 2	S=(1, 0)	A=↓	R=-1.0	S'=(2, 0)	
t= 3	S=(2, 0)	A=↓	R=-1.0	S'=(3, 0)	
t= 4	S=(3, 0)	A=←	R=-1.0	S'=(3, 0)	
t= 5	S=(3, 0)	A=→	R=-1.0	S'=(3, 1)	
t= 6	S=(3, 1)	A=→	R=-1.0	S'=(3, 2)	
t= 7	S=(3, 2)	A=←	R=-1.0	S'=(3, 1)	
t= 8	S=(3, 1)	A=↓	R=-1.0	S'=(3, 1)	
t= 9	S=(3, 1)	A=↓	R=-1.0	S'=(3, 1)	
t=10	S=(3, 1)	A=→	R=-1.0	S'=(3, 2)	
t=11	S=(3, 2)	A=↑	R=-1.0	S'=(2, 2)	
t=12	S=(2, 2)	A=↑	R=-1.0	S'=(1, 2)	
t=13	S=(1, 2)	A=↑	R=-1.0	S'=(0, 2)	
t=14	S=(0, 2)	A=↓	R=-1.0	S'=(1, 2)	
t=15	S=(1, 2)	A=↓	R=-1.0	S'=(2, 2)	
t=16	S=(2, 2)	A=→	R=-5.0	S'=(2, 3)	x TRAMPA

Pasos totales : 17

Recompensa acumulada : -21.0
Retorno G_0 ($\gamma=0.99$): -19.1115

► Episodio 5/5

INICIO DEL EPISODIO Estado inicial: (0, 0)

```
t= 0 S=(0, 0) A=↑ R=-1.0 S'=(0, 0)
t= 1 S=(0, 0) A=→ R=-1.0 S'=(0, 1)
t= 2 S=(0, 1) A=→ R=-1.0 S'=(0, 2)
t= 3 S=(0, 2) A=→ R=-1.0 S'=(0, 3)
t= 4 S=(0, 3) A=← R=-1.0 S'=(0, 2)

t= 5 S=(0, 2) A=↑ R=-1.0 S'=(0, 2)
t= 6 S=(0, 2) A=↑ R=-1.0 S'=(0, 2)
t= 7 S=(0, 2) A=← R=-1.0 S'=(0, 1)
t= 8 S=(0, 1) A=← R=-1.0 S'=(0, 0)
t= 9 S=(0, 0) A=↑ R=-1.0 S'=(0, 0)
t=10 S=(0, 0) A=← R=-1.0 S'=(0, 0)
t=11 S=(0, 0) A=→ R=-1.0 S'=(0, 1)
t=12 S=(0, 1) A=↓ R=-5.0 S'=(1, 1) x TRAMPA
```

Pasos totales : 13
Recompensa acumulada : -17.0
Retorno G_0 ($\gamma=0.99$): -15.7934

RESUMEN DEL EXPERIMENTO

Retorno promedio G_0 : -20.6194
Retorno máximo : -12.3393
Retorno mínimo : -32.7771

Observación: la variabilidad entre episodios ilustra por qué el objetivo de RL es el RETORNO ESPERADO, no un retorno fijo.

1. Cambio del factor de descuento GAMMA

Se ejecutó el experimento utilizando los valores de $\text{GAMMA} = 0.5$ y $\text{GAMMA} = 0.99$. Como referencia, el valor original era $\text{GAMMA} = 0.95$. Debido a que se utilizó la misma semilla aleatoria (`random.seed(42)`), las acciones y trayectorias fueron iguales en las tres ejecuciones; únicamente cambió la forma en que se calculó el retorno descontado.

Output con GAMMA = 0.95

RESUMEN DEL EXPERIMENTO

Retorno promedio G_0 : -13.5606
Retorno máximo : -10.0487
Retorno mínimo : -17.5702

Output con GAMMA = 0.5

RESUMEN DEL EXPERIMENTO

Retorno promedio G_0 : -2.0025
Retorno máximo : -2.0000
Retorno mínimo : -2.0117

Output con GAMMA = 0.99

RESUMEN DEL EXPERIMENTO

Retorno promedio G_0 : -20.6194
Retorno máximo : -12.3393
Retorno mínimo : -32.7771

Comparación de resultados

GAMMA	Retorno promedio (G_0)
0.50	-2.0025
0.95	-13.5606
0.99	-20.6194

Observación

Cuando GAMMA disminuye a 0.5, las recompensas futuras pierden importancia rápidamente, por lo que el retorno depende principalmente de los primeros pasos y se mantiene cerca de -2. En cambio, con GAMMA = 0.99, las recompensas y penalizaciones futuras conservan casi todo su peso, haciendo que los numerosos costos de -1 produzcan un retorno promedio mucho más negativo. Esto demuestra que el factor de descuento determina cuánto influyen las consecuencias de largo plazo en el retorno del agente.

Código

```
"""
Experimentos de GridWorld 4x4

Escenario original:
- Trampas en (1, 1) y (2, 3)
- Recompensa por paso: -1

Pregunta 2:
- Agregar una nueva trampa en (0, 3)

Pregunta 3:
- Cambiar la recompensa por paso de -1 a -0.1
"""

import numpy as np
import random
from collections import Counter

# =====
# CONSTANTES GENERALES
# =====

GRID_SIZE = 4
GOAL_STATE = (3, 3)

ORIGINAL_TRAP_STATES = {(1, 1), (2, 3)}
NEW_TRAP_STATE = (0, 3)

MAX_STEPS = 50
GAMMA = 0.95
N_EPISODES = 5
SEED = 42

# Cambia a False si solamente quieres imprimir los resúmenes.
SHOW_FULL_TRAJECTORIES = True

# =====
# ACCIONES
# =====

ACTIONS = {
    0: (-1, 0), # Arriba
    1: (1, 0), # Abajo
    2: (0, -1), # Izquierda
    3: (0, 1), # Derecha
}

ACTION_NAMES = {
    0: "↑",
    1: "↓",
    2: "←",
    3: "→",
}

# =====
# ENTORNO
# =====

class GridWorldEnv:
    """
    Entorno GridWorld configurable.

    Permite cambiar:
    - Las posiciones de las trampas.
    - La recompensa recibida por cada paso normal.
    """

    def __init__(self, trap_states, step_reward=-1.0):
        self.trap_states = set(trap_states)
        self.step_reward = float(step_reward)
        self.state = None
        self.done = False

    def reset(self):
        """Reinicia el entorno y devuelve el estado inicial."""
        self.state = (0, 0)
        self.done = False
        return self.state

    def step(self, action):
        """
        Ejecuta una acción y devuelve:

        next_state, reward, done
        """

        if self.done:
            raise RuntimeError(
                "El episodio ya terminó. Debes llamar reset()."
            )

        dr, dc = ACTIONS[action]
```

```

        row, column = self.state

        # Evita que el agente salga del grid.
        new_row = max(
            0,
            min(GRID_SIZE - 1, row + dr)
        )

        new_column = max(
            0,
            min(GRID_SIZE - 1, column + dc)
        )

        next_state = (new_row, new_column)

        # Recompensas y estados terminales.
        if next_state == GOAL_STATE:
            reward = 10.0
            self.done = True
        elif next_state in self.trap_states:
            reward = -5.0
            self.done = True
        else:
            reward = self.step_reward

        self.state = next_state

        return next_state, reward, self.done

# =====
# AGENTE
# =====

class RandomAgent:
    """
    Agente con política aleatoria uniforme.

    No aprende ni modifica su política.
    """

    def __init__(self, action_space_size):
        self.n_actions = action_space_size

    def select_action(self, state):
        """Selecciona una acción aleatoria."""
        return random.randint(0, self.n_actions - 1)

# =====
# EJECUCIÓN DE UN EPISODIO
# =====

def run_episode(
    env,
    agent,
    gamma=GAMMA,
    max_steps=MAX_STEPS,
    verbose=True
):
    """
    Ejecuta un episodio y calcula el retorno descontado G_0.
    """

    state = env.reset()

    trajectory = []
    total_reward = 0.0

    if verbose:
        print(f"\n{'=' * 55}")
        print(f"INICIO DEL EPISODIO | Estado inicial: {state}")
        print(f"{'-' * 55}")

    for t in range(max_steps):
        # El agente selecciona una acción.
        action = agent.select_action(state)

        # El entorno ejecuta la transición.
        next_state, reward, done = env.step(action)

        trajectory.append(
            (state, action, reward, next_state)
        )

        total_reward += reward

        if verbose:
            if next_state == GOAL_STATE:
                status = "✓ META"

            elif next_state in env.trap_states:
                status = "x TRAMPA"

            else:
                status = ""

        print(
            f"t={t:2d} "
            f"S={state} "
            f"A={ACTION_NAMES[action]} "
            f"R={reward:+.1f} "
            f"S'={next_state} "
            f"{status}"
        )
    )

```



```

        state = next_state

    if done:
        break

# Cálculo del retorno descontado:
#  $G_t = R_{(t+1)} + \gamma * G_{(t+1)}$ 

G = 0.0
returns = []

for _, _, reward, _ in reversed(trajecory):
    G = reward + gamma * G
    returns.insert(0, G)

final_state = trajectory[-1][3]

if final_state == GOAL_STATE:
    outcome = "META"

elif final_state in env.trap_states:
    outcome = f"TRAMPA {final_state}"

else:
    outcome = "LÍMITE DE PASOS"

if verbose:
    print(f"{'-' * 55}")
    print(f" Pasos totales           : {len(trajectory)}")
    print(f" Resultado final             : {outcome}")
    print(f" Recompensa acumulada       : {total_reward:+.4f}")
    print(f" Retorno  $G_0$  ( $\gamma$ ={gamma}): {returns[0]:.4f}")
    print(f"{'=' * 55}")

return {
    "trajectory": trajectory,
    "returns": returns,
    "G0": returns[0],
    "total_reward": total_reward,
    "steps": len(trajectory),
    "outcome": outcome,
}

# =====
# EJECUCIÓN DE VARIOS EPISODIOS
# =====

def run_experiment(
    experiment_name,
    trap_states,
    step_reward,
    n_episodes=N_EPISODES,
    verbose=SHOW_FULL_TRAJECTORIES
):
    """
    Ejecuta varios episodios bajo una configuración determinada.
    """

    # Reiniciar las semillas permite comparar los experimentos
    # usando la misma secuencia aleatoria inicial.
    random.seed(SEED)
    np.random.seed(SEED)

    env = GridWorldEnv(
        trap_states=trap_states,
        step_reward=step_reward,
    )

    agent = RandomAgent(
        action_space_size=len(ACTIONS)
    )

    print("\n\n" + "█" * 65)
    print(experiment_name)
    print("█" * 65)

    print(f"Trampas           : {sorted(trap_states)}")
    print(f"Recompensa por paso : {step_reward}")
    print(f"Factor de descuento : {GAMMA}")
    print(f"Número de episodios : {n_episodes}")

    results = []

    for episode in range(n_episodes):

        if verbose:
            print(
                f"\n► Episodio "
                f"{episode + 1}/{n_episodes}"
            )

        result = run_episode(
            env=env,
            agent=agent,
            gamma=GAMMA,
            max_steps=MAX_STEPS,
            verbose=verbose,
        )
        results.append(result)

    g0_values = [
        result["G0"]
        for result in results
    ]

    accumulated_rewards = [

```

```

        result["total_reward"]
        for result in results
    ]

    steps = [
        result["steps"]
        for result in results
    ]

    outcomes = [
        result["outcome"]
        for result in results
    ]

    print("\n" + "=" * 65)
    print("RESUMEN DEL EXPERIMENTO")
    print("=" * 65)

    print("\nRetornos G_0 por episodio:")

    for index, value in enumerate(g0_values, start=1):
        print(
            f"    Episodio {index}: {value:.4f}"
        )

    print("\nEstadísticas de los retornos:")

    print(
        f"    Retorno promedio G_0 : "
        f"{np.mean(g0_values):.4f}"
    )

    print(
        f"    Desviación estándar : "
        f"{np.std(g0_values):.4f}"
    )

    print(
        f"    Retorno máximo      : "
        f"{max(g0_values):.4f}"
    )

    print(
        f"    Retorno mínimo       : "
        f"{min(g0_values):.4f}"
    )

    print(
        f"    Promedio de pasos    : "
        f"{np.mean(steps):.2f}"
    )

    print("\nDistribución de resultados:")

    outcome_distribution = Counter(outcomes)

    for outcome, quantity in outcome_distribution.items():
        print(
            f"    {outcome}: {quantity} episodio(s)"
        )

    print("=" * 65)

    return {
        "name": experiment_name,
        "results": results,
        "g0_values": g0_values,
        "accumulated_rewards": accumulated_rewards,
        "steps": steps,
        "outcomes": outcomes,
        "mean_G0": float(np.mean(g0_values)),
        "std_G0": float(np.std(g0_values)),
    }

# =====
# COMPARACIÓN ENTRE EXPERIMENTOS
# =====

def compare_experiments(
    baseline,
    trap_experiment,
    step_reward_experiment
):
    """
    Imprime una comparación directa para responder
    las preguntas 2 y 3.
    """

    print("\n\n" + "■" * 65)
    print("COMPARACION FINAL")
    print("■" * 65)

    # -----
    # PREGUNTA 2
    # -----

    print("\nPREGUNTA 2: NUEVA TRAMPA EN (0, 3)")
    print("-" * 65)

    print(
        f"Promedio original      : "
        f"{baseline['mean_G0']:.4f}"
    )

    print(
        f"Promedio con nueva trampa: "

```

```

        f"{trap_experiment['mean_G0']:.4f}"
    )

    print(
        f"Desviación original      : "
        f"{baseline['std_G0']:.4f}"
    )

    print(
        f"Desviación nueva trampa : "
        f"{trap_experiment['std_G0']:.4f}"
    )

    print("\nRetornos originales:")

    print(
        [
            round(value, 4)
            for value in baseline["g0_values"]
        ]
    )

    print("\nRetornos con la trampa (0, 3):")

    print(
        [
            round(value, 4)
            for value in trap_experiment["g0_values"]
        ]
    )

    print("\nResultados terminales originales:")

    print(
        dict(Counter(baseline["outcomes"]))
    )

    print("\nResultados terminales con nueva trampa:")

    print(
        dict(Counter(trap_experiment["outcomes"]))
    )

    new_trap_hits = sum(
        outcome == "TRAMPA (0, 3)"
        for outcome in trap_experiment["outcomes"]
    )

    print(
        f"\nVeces que el agente cayó en la nueva trampa: "
        f"{new_trap_hits}"
    )

    # -----
    # PREGUNTA 3
    # -----

    print("\n\nPREGUNTA 3: RECOMPENSA POR PASO DE -0.1")
    print("-" * 65)

    print(
        f"Promedio con paso -1.0 : "
        f"{baseline['mean_G0']:.4f}"
    )

    print(
        f"Promedio con paso -0.1 : "
        f"{step_reward_experiment['mean_G0']:.4f}"
    )

    print("\nRetornos con recompensa por paso de -1.0:")

    print(
        [
            round(value, 4)
            for value in baseline["g0_values"]
        ]
    )

    print("\nRetornos con recompensa por paso de -0.1:")

    print(
        [
            round(value, 4)
            for value in step_reward_experiment["g0_values"]
        ]
    )

    print(
        f"\nPromedio de pasos con recompensa -1.0: "
        f"{np.mean(baseline['steps']):.2f}"
    )

    print(
        f"Promedio de pasos con recompensa -0.1: "
        f"{np.mean(step_reward_experiment['steps']):.2f}"
    )

    # Compara estados y acciones, ignorando las recompensas.
    same_behavior = True

    for baseline_result, modified_result in zip(
        baseline["results"],
        step_reward_experiment["results"],
    ):

        baseline_path = [
            (state, action, next_state)

```

```

        for state, action, _, next_state
        in baseline_result["trajectory"]
    ]

    modified_path = [
        (state, action, next_state)
        for state, action, _, next_state
        in modified_result["trajectory"]
    ]

    if baseline_path != modified_path:
        same_behavior = False
        break

print(
    "\nLas acciones y trayectorias fueron iguales "
    "al cambiar la recompensa?"
)

print(
    f" {same_behavior}"
)

print(
    "\nNota: el agente utiliza una política aleatoria y no aprende. "
    "Por eso, cambiar la recompensa modifica los retornos, pero no "
    "cambia la forma en que selecciona sus acciones."
)

print("█" * 65)

# =====
# PROGRAMA PRINCIPAL
# =====

if __name__ == "__main__":

    # -----
    # ESCENARIO ORIGINAL
    # -----

    baseline_results = run_experiment(
        experiment_name=(
            "ESCENARIO ORIGINAL: "
            "TRAMPAS ORIGINALES Y RECOMPENSA -1"
        ),
        trap_states=ORIGINAL_TRAP_STATES,
        step_reward=-1.0,
    )

    # -----
    # PREGUNTA 2
    # Nueva trampa en (0, 3)
    # -----

    traps_question_2 = (
        ORIGINAL_TRAP_STATES | {NEW_TRAP_STATE}
    )

    trap_results = run_experiment(
        experiment_name=(
            "PREGUNTA 2: "
            "NUEVA TRAMPA EN LA POSICIÓN (0, 3)"
        ),
        trap_states=traps_question_2,
        step_reward=-1.0,
    )

    # -----
    # PREGUNTA 3
    # Recompensa por paso de -0.1
    # -----

    step_reward_results = run_experiment(
        experiment_name=(
            "PREGUNTA 3: "
            "RECOMPENSA POR PASO CAMBIADA A -0.1"
        ),
        trap_states=ORIGINAL_TRAP_STATES,
        step_reward=-0.1,
    )

    # -----
    # COMPARACIÓN AUTOMÁTICA
    # -----

    compare_experiments(
        baseline=baseline_results,
        trap_experiment=trap_results,
        step_reward_experiment=step_reward_results,
    )

```

Output

```

████████████████████████████████████████████████████████████████████████████████
ESCENARIO ORIGINAL: TRAMPAS ORIGINALES Y RECOMPENSA -1
████████████████████████████████████████████████████████████████████████████████

```

```

Trampas           : [(1, 1), (2, 3)]
Recompensa por paso : -1.0
Factor de descuento : 0.95
Número de episodios : 5

```

► Episodio 1/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

t= 0	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)
t= 1	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)
t= 2	S=(0, 0)	A=←	R=-1.0	S'=(0, 0)
t= 3	S=(0, 0)	A=↓	R=-1.0	S'=(1, 0)
t= 4	S=(1, 0)	A=↓	R=-1.0	S'=(2, 0)
t= 5	S=(2, 0)	A=↓	R=-1.0	S'=(3, 0)
t= 6	S=(3, 0)	A=↑	R=-1.0	S'=(2, 0)
t= 7	S=(2, 0)	A=↑	R=-1.0	S'=(1, 0)
t= 8	S=(1, 0)	A=→	R=-5.0	S'=(1, 1) x TRAMPA

Pasos totales	: 9
Resultado final	: TRAMPA (1, 1)
Recompensa acumulada	: -13.0000
Retorno G_0 (γ=0.95):	-10.0487

► Episodio 2/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

t= 0	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)
t= 1	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)
t= 2	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)
t= 3	S=(0, 0)	A=↓	R=-1.0	S'=(1, 0)
t= 4	S=(1, 0)	A=↓	R=-1.0	S'=(2, 0)
t= 5	S=(2, 0)	A=↑	R=-1.0	S'=(1, 0)
t= 6	S=(1, 0)	A=↓	R=-1.0	S'=(2, 0)
t= 7	S=(2, 0)	A=→	R=-1.0	S'=(2, 1)
t= 8	S=(2, 1)	A=↓	R=-1.0	S'=(3, 1)
t= 9	S=(3, 1)	A=→	R=-1.0	S'=(3, 2)
t=10	S=(3, 2)	A=←	R=-1.0	S'=(3, 1)
t=11	S=(3, 1)	A=↑	R=-1.0	S'=(2, 1)
t=12	S=(2, 1)	A=↓	R=-1.0	S'=(3, 1)
t=13	S=(3, 1)	A=→	R=-1.0	S'=(3, 2)
t=14	S=(3, 2)	A=←	R=-1.0	S'=(3, 1)
t=15	S=(3, 1)	A=←	R=-1.0	S'=(3, 0)
t=16	S=(3, 0)	A=↓	R=-1.0	S'=(3, 0)
t=17	S=(3, 0)	A=↓	R=-1.0	S'=(3, 0)
t=18	S=(3, 0)	A=←	R=-1.0	S'=(3, 0)
t=19	S=(3, 0)	A=↑	R=-1.0	S'=(2, 0)
t=20	S=(2, 0)	A=↑	R=-1.0	S'=(1, 0)
t=21	S=(1, 0)	A=→	R=-5.0	S'=(1, 1) x TRAMPA

Pasos totales	: 22
Resultado final	: TRAMPA (1, 1)
Recompensa acumulada	: -26.0000
Retorno G_0 (γ=0.95):	-14.8916

► Episodio 3/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

t= 0	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)
t= 1	S=(0, 0)	A=←	R=-1.0	S'=(0, 0)
t= 2	S=(0, 0)	A=←	R=-1.0	S'=(0, 0)
t= 3	S=(0, 0)	A=←	R=-1.0	S'=(0, 0)
t= 4	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)
t= 5	S=(0, 0)	A=→	R=-1.0	S'=(0, 1)
t= 6	S=(0, 1)	A=↑	R=-1.0	S'=(0, 1)
t= 7	S=(0, 1)	A=→	R=-1.0	S'=(0, 2)
t= 8	S=(0, 2)	A=↑	R=-1.0	S'=(0, 2)
t= 9	S=(0, 2)	A=←	R=-1.0	S'=(0, 1)
t=10	S=(0, 1)	A=←	R=-1.0	S'=(0, 0)
t=11	S=(0, 0)	A=↓	R=-1.0	S'=(1, 0)
t=12	S=(1, 0)	A=↑	R=-1.0	S'=(0, 0)
t=13	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)
t=14	S=(0, 0)	A=↓	R=-1.0	S'=(1, 0)
t=15	S=(1, 0)	A=←	R=-1.0	S'=(1, 0)
t=16	S=(1, 0)	A=↑	R=-1.0	S'=(0, 0)
t=17	S=(0, 0)	A=↓	R=-1.0	S'=(1, 0)
t=18	S=(1, 0)	A=↑	R=-1.0	S'=(0, 0)
t=19	S=(0, 0)	A=→	R=-1.0	S'=(0, 1)
t=20	S=(0, 1)	A=←	R=-1.0	S'=(0, 0)
t=21	S=(0, 0)	A=→	R=-1.0	S'=(0, 1)
t=22	S=(0, 1)	A=←	R=-1.0	S'=(0, 0)
t=23	S=(0, 0)	A=↓	R=-1.0	S'=(1, 0)
t=24	S=(1, 0)	A=←	R=-1.0	S'=(1, 0)
t=25	S=(1, 0)	A=←	R=-1.0	S'=(1, 0)
t=26	S=(1, 0)	A=↓	R=-1.0	S'=(2, 0)
t=27	S=(2, 0)	A=←	R=-1.0	S'=(2, 0)
t=28	S=(2, 0)	A=↑	R=-1.0	S'=(1, 0)
t=29	S=(1, 0)	A=↓	R=-1.0	S'=(2, 0)
t=30	S=(2, 0)	A=↓	R=-1.0	S'=(3, 0)
t=31	S=(3, 0)	A=↓	R=-1.0	S'=(3, 0)
t=32	S=(3, 0)	A=→	R=-1.0	S'=(3, 1)
t=33	S=(3, 1)	A=→	R=-1.0	S'=(3, 2)
t=34	S=(3, 2)	A=←	R=-1.0	S'=(3, 1)
t=35	S=(3, 1)	A=↓	R=-1.0	S'=(3, 1)
t=36	S=(3, 1)	A=←	R=-1.0	S'=(3, 0)
t=37	S=(3, 0)	A=↑	R=-1.0	S'=(2, 0)
t=38	S=(2, 0)	A=↓	R=-1.0	S'=(3, 0)
t=39	S=(3, 0)	A=↑	R=-1.0	S'=(2, 0)
t=40	S=(2, 0)	A=←	R=-1.0	S'=(2, 0)
t=41	S=(2, 0)	A=→	R=-1.0	S'=(2, 1)
t=42	S=(2, 1)	A=←	R=-1.0	S'=(2, 0)
t=43	S=(2, 0)	A=↑	R=-1.0	S'=(1, 0)
t=44	S=(1, 0)	A=↓	R=-1.0	S'=(2, 0)
t=45	S=(2, 0)	A=←	R=-1.0	S'=(2, 0)
t=46	S=(2, 0)	A=↓	R=-1.0	S'=(3, 0)
t=47	S=(3, 0)	A=→	R=-1.0	S'=(3, 1)
t=48	S=(3, 1)	A=→	R=-1.0	S'=(3, 2)
t=49	S=(3, 2)	A=→	R=+10.0	S'=(3, 3) ✓ META

Pasos totales	: 50
Resultado final	: META
Recompensa acumulada	: -39.0000

Retorno G_0 ($\gamma=0.95$): -17.5702

► Episodio 4/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

t= 0	S=(0, 0)	A=↓	R=-1.0	S'=(1, 0)
t= 1	S=(1, 0)	A=←	R=-1.0	S'=(1, 0)
t= 2	S=(1, 0)	A=↓	R=-1.0	S'=(2, 0)
t= 3	S=(2, 0)	A=↓	R=-1.0	S'=(3, 0)
t= 4	S=(3, 0)	A=←	R=-1.0	S'=(3, 0)
t= 5	S=(3, 0)	A=→	R=-1.0	S'=(3, 1)
t= 6	S=(3, 1)	A=→	R=-1.0	S'=(3, 2)
t= 7	S=(3, 2)	A=←	R=-1.0	S'=(3, 1)
t= 8	S=(3, 1)	A=↓	R=-1.0	S'=(3, 1)
t= 9	S=(3, 1)	A=↓	R=-1.0	S'=(3, 1)
t=10	S=(3, 1)	A=→	R=-1.0	S'=(3, 2)
t=11	S=(3, 2)	A=↑	R=-1.0	S'=(2, 2)
t=12	S=(2, 2)	A=↑	R=-1.0	S'=(1, 2)
t=13	S=(1, 2)	A=↑	R=-1.0	S'=(0, 2)
t=14	S=(0, 2)	A=↓	R=-1.0	S'=(1, 2)
t=15	S=(1, 2)	A=↓	R=-1.0	S'=(2, 2)
t=16	S=(2, 2)	A=→	R=-5.0	S'=(2, 3) x TRAMPA

Pasos totales : 17
Resultado final : TRAMPA (2, 3)
Recompensa acumulada : -21.0000
Retorno G_0 ($\gamma=0.95$): -13.3981

► Episodio 5/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

t= 0	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)
t= 1	S=(0, 0)	A=→	R=-1.0	S'=(0, 1)
t= 2	S=(0, 1)	A=→	R=-1.0	S'=(0, 2)
t= 3	S=(0, 2)	A=→	R=-1.0	S'=(0, 3)
t= 4	S=(0, 3)	A=←	R=-1.0	S'=(0, 2)
t= 5	S=(0, 2)	A=↑	R=-1.0	S'=(0, 2)
t= 6	S=(0, 2)	A=↑	R=-1.0	S'=(0, 2)
t= 7	S=(0, 2)	A=←	R=-1.0	S'=(0, 1)
t= 8	S=(0, 1)	A=←	R=-1.0	S'=(0, 0)
t= 9	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)
t=10	S=(0, 0)	A=←	R=-1.0	S'=(0, 0)
t=11	S=(0, 0)	A=→	R=-1.0	S'=(0, 1)
t=12	S=(0, 1)	A=↓	R=-5.0	S'=(1, 1) x TRAMPA

Pasos totales : 13
Resultado final : TRAMPA (1, 1)
Recompensa acumulada : -17.0000
Retorno G_0 ($\gamma=0.95$): -11.8946

RESUMEN DEL EXPERIMENTO

Retornos G_0 por episodio:

Episodio 1: -10.0487
Episodio 2: -14.8916
Episodio 3: -17.5702
Episodio 4: -13.3981
Episodio 5: -11.8946

Estadísticas de los retornos:

Retorno promedio G_0 : -13.5606
Desviación estándar : 2.5684
Retorno máximo : -10.0487
Retorno mínimo : -17.5702
Promedio de pasos : 22.20

Distribución de resultados:

TRAMPA (1, 1): 3 episodio(s)
META: 1 episodio(s)
TRAMPA (2, 3): 1 episodio(s)

PREGUNTA 2: NUEVA TRAMPA EN LA POSICIÓN (0, 3)

Trampas : [(0, 3), (1, 1), (2, 3)]
Recompensa por paso : -1.0
Factor de descuento : 0.95
Número de episodios : 5

► Episodio 1/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

t= 0	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)
t= 1	S=(0, 0)	A=↑	R=-1.0	S'=(0, 0)
t= 2	S=(0, 0)	A=←	R=-1.0	S'=(0, 0)
t= 3	S=(0, 0)	A=↓	R=-1.0	S'=(1, 0)
t= 4	S=(1, 0)	A=↓	R=-1.0	S'=(2, 0)
t= 5	S=(2, 0)	A=↓	R=-1.0	S'=(3, 0)
t= 6	S=(3, 0)	A=↑	R=-1.0	S'=(2, 0)
t= 7	S=(2, 0)	A=↑	R=-1.0	S'=(1, 0)
t= 8	S=(1, 0)	A=→	R=-5.0	S'=(1, 1) x TRAMPA

Pasos totales : 9
Resultado final : TRAMPA (1, 1)
Recompensa acumulada : -13.0000
Retorno G_0 ($\gamma=0.95$): -10.0487

► Episodio 2/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

```

t= 0 S=(0, 0) A=↑ R=-1.0 S'=(0, 0)
t= 1 S=(0, 0) A=↑ R=-1.0 S'=(0, 0)
t= 2 S=(0, 0) A=↑ R=-1.0 S'=(0, 0)
t= 3 S=(0, 0) A=↓ R=-1.0 S'=(1, 0)
t= 4 S=(1, 0) A=↓ R=-1.0 S'=(2, 0)
t= 5 S=(2, 0) A=↑ R=-1.0 S'=(1, 0)
t= 6 S=(1, 0) A=↓ R=-1.0 S'=(2, 0)
t= 7 S=(2, 0) A=→ R=-1.0 S'=(2, 1)
t= 8 S=(2, 1) A=↓ R=-1.0 S'=(3, 1)
t= 9 S=(3, 1) A=→ R=-1.0 S'=(3, 2)
t=10 S=(3, 2) A=← R=-1.0 S'=(3, 1)
t=11 S=(3, 1) A=↑ R=-1.0 S'=(2, 1)
t=12 S=(2, 1) A=↓ R=-1.0 S'=(3, 1)
t=13 S=(3, 1) A=→ R=-1.0 S'=(3, 2)
t=14 S=(3, 2) A=← R=-1.0 S'=(3, 1)
t=15 S=(3, 1) A=← R=-1.0 S'=(3, 0)
t=16 S=(3, 0) A=↓ R=-1.0 S'=(3, 0)
t=17 S=(3, 0) A=↓ R=-1.0 S'=(3, 0)
t=18 S=(3, 0) A=← R=-1.0 S'=(3, 0)
t=19 S=(3, 0) A=↑ R=-1.0 S'=(2, 0)
t=20 S=(2, 0) A=↑ R=-1.0 S'=(1, 0)
t=21 S=(1, 0) A=→ R=-5.0 S'=(1, 1) x TRAMPA

```

```

Pasos totales      : 22
Resultado final    : TRAMPA (1, 1)
Recompensa acumulada : -26.0000
Retorno G_0 (γ=0.95): -14.8916

```

► Episodio 3/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

```

t= 0 S=(0, 0) A=↑ R=-1.0 S'=(0, 0)
t= 1 S=(0, 0) A=← R=-1.0 S'=(0, 0)
t= 2 S=(0, 0) A=← R=-1.0 S'=(0, 0)
t= 3 S=(0, 0) A=← R=-1.0 S'=(0, 0)
t= 4 S=(0, 0) A=↑ R=-1.0 S'=(0, 0)
t= 5 S=(0, 0) A=→ R=-1.0 S'=(0, 1)

t= 6 S=(0, 1) A=↑ R=-1.0 S'=(0, 1)
t= 7 S=(0, 1) A=→ R=-1.0 S'=(0, 2)
t= 8 S=(0, 2) A=↑ R=-1.0 S'=(0, 2)
t= 9 S=(0, 2) A=← R=-1.0 S'=(0, 1)
t=10 S=(0, 1) A=← R=-1.0 S'=(0, 0)
t=11 S=(0, 0) A=↓ R=-1.0 S'=(1, 0)
t=12 S=(1, 0) A=↑ R=-1.0 S'=(0, 0)
t=13 S=(0, 0) A=↑ R=-1.0 S'=(0, 0)
t=14 S=(0, 0) A=↓ R=-1.0 S'=(1, 0)
t=15 S=(1, 0) A=← R=-1.0 S'=(1, 0)
t=16 S=(1, 0) A=↑ R=-1.0 S'=(0, 0)
t=17 S=(0, 0) A=↓ R=-1.0 S'=(1, 0)
t=18 S=(1, 0) A=↑ R=-1.0 S'=(0, 0)
t=19 S=(0, 0) A=→ R=-1.0 S'=(0, 1)
t=20 S=(0, 1) A=← R=-1.0 S'=(0, 0)
t=21 S=(0, 0) A=→ R=-1.0 S'=(0, 1)
t=22 S=(0, 1) A=← R=-1.0 S'=(0, 0)
t=23 S=(0, 0) A=↓ R=-1.0 S'=(1, 0)
t=24 S=(1, 0) A=← R=-1.0 S'=(1, 0)
t=25 S=(1, 0) A=← R=-1.0 S'=(1, 0)
t=26 S=(1, 0) A=↓ R=-1.0 S'=(2, 0)
t=27 S=(2, 0) A=← R=-1.0 S'=(2, 0)
t=28 S=(2, 0) A=↑ R=-1.0 S'=(1, 0)
t=29 S=(1, 0) A=↓ R=-1.0 S'=(2, 0)
t=30 S=(2, 0) A=↓ R=-1.0 S'=(3, 0)
t=31 S=(3, 0) A=↓ R=-1.0 S'=(3, 0)
t=32 S=(3, 0) A=→ R=-1.0 S'=(3, 1)
t=33 S=(3, 1) A=→ R=-1.0 S'=(3, 2)
t=34 S=(3, 2) A=← R=-1.0 S'=(3, 1)
t=35 S=(3, 1) A=↓ R=-1.0 S'=(3, 1)
t=36 S=(3, 1) A=← R=-1.0 S'=(3, 0)
t=37 S=(3, 0) A=↑ R=-1.0 S'=(2, 0)
t=38 S=(2, 0) A=↓ R=-1.0 S'=(3, 0)
t=39 S=(3, 0) A=↑ R=-1.0 S'=(2, 0)
t=40 S=(2, 0) A=← R=-1.0 S'=(2, 0)
t=41 S=(2, 0) A=→ R=-1.0 S'=(2, 1)
t=42 S=(2, 1) A=← R=-1.0 S'=(2, 0)
t=43 S=(2, 0) A=↑ R=-1.0 S'=(1, 0)
t=44 S=(1, 0) A=↓ R=-1.0 S'=(2, 0)
t=45 S=(2, 0) A=← R=-1.0 S'=(2, 0)

t=46 S=(2, 0) A=↓ R=-1.0 S'=(3, 0)
t=47 S=(3, 0) A=→ R=-1.0 S'=(3, 1)
t=48 S=(3, 1) A=→ R=-1.0 S'=(3, 2)
t=49 S=(3, 2) A=→ R=+10.0 S'=(3, 3) ✓ META

```

```

Pasos totales      : 50
Resultado final    : META
Recompensa acumulada : -39.0000
Retorno G_0 (γ=0.95): -17.5702

```

► Episodio 4/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

```

t= 0 S=(0, 0) A=↓ R=-1.0 S'=(1, 0)
t= 1 S=(1, 0) A=← R=-1.0 S'=(1, 0)
t= 2 S=(1, 0) A=↓ R=-1.0 S'=(2, 0)
t= 3 S=(2, 0) A=↓ R=-1.0 S'=(3, 0)
t= 4 S=(3, 0) A=← R=-1.0 S'=(3, 0)
t= 5 S=(3, 0) A=→ R=-1.0 S'=(3, 1)
t= 6 S=(3, 1) A=→ R=-1.0 S'=(3, 2)
t= 7 S=(3, 2) A=← R=-1.0 S'=(3, 1)

```

```
t= 8 S=(3, 1) A=↓ R=-1.0 S'=(3, 1)
t= 9 S=(3, 1) A=↓ R=-1.0 S'=(3, 1)
t=10 S=(3, 1) A=→ R=-1.0 S'=(3, 2)
t=11 S=(3, 2) A=↑ R=-1.0 S'=(2, 2)
t=12 S=(2, 2) A=↑ R=-1.0 S'=(1, 2)
t=13 S=(1, 2) A=↑ R=-1.0 S'=(0, 2)
t=14 S=(0, 2) A=↓ R=-1.0 S'=(1, 2)
t=15 S=(1, 2) A=↓ R=-1.0 S'=(2, 2)
t=16 S=(2, 2) A=→ R=-5.0 S'=(2, 3) x TRAMPA

Pasos totales : 17
Resultado final : TRAMPA (2, 3)
Recompensa acumulada : -21.0000
Retorno G_0 (γ=0.95): -13.3981
```

► Episodio 5/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

```
t= 0 S=(0, 0) A=↑ R=-1.0 S'=(0, 0)
t= 1 S=(0, 0) A=→ R=-1.0 S'=(0, 1)
t= 2 S=(0, 1) A=→ R=-1.0 S'=(0, 2)
t= 3 S=(0, 2) A=→ R=-5.0 S'=(0, 3) x TRAMPA

Pasos totales : 4
Resultado final : TRAMPA (0, 3)
Recompensa acumulada : -8.0000
Retorno G_0 (γ=0.95): -7.1394
```

RESUMEN DEL EXPERIMENTO

Retornos G_0 por episodio:

```
Episodio 1: -10.0487
Episodio 2: -14.8916
Episodio 3: -17.5702
Episodio 4: -13.3981
Episodio 5: -7.1394
```

Estadísticas de los retornos:

```
Retorno promedio G_0 : -12.6096
Desviación estándar : 3.6584
Retorno máximo : -7.1394
Retorno mínimo : -17.5702
Promedio de pasos : 20.40
```

Distribución de resultados:

```
TRAMPA (1, 1): 2 episodio(s)
META: 1 episodio(s)
TRAMPA (2, 3): 1 episodio(s)
TRAMPA (0, 3): 1 episodio(s)
```

PREGUNTA 3: RECOMPENSA POR PASO CAMBIADA A -0.1

```
Trampas : [(1, 1), (2, 3)]
Recompensa por paso : -0.1
Factor de descuento : 0.95
Número de episodios : 5
```

► Episodio 1/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

```
t= 0 S=(0, 0) A=↑ R=-0.1 S'=(0, 0)
t= 1 S=(0, 0) A=↑ R=-0.1 S'=(0, 0)
t= 2 S=(0, 0) A=← R=-0.1 S'=(0, 0)
t= 3 S=(0, 0) A=↓ R=-0.1 S'=(1, 0)
t= 4 S=(1, 0) A=↓ R=-0.1 S'=(2, 0)
t= 5 S=(2, 0) A=↓ R=-0.1 S'=(3, 0)
t= 6 S=(3, 0) A=↑ R=-0.1 S'=(2, 0)
t= 7 S=(2, 0) A=↑ R=-0.1 S'=(1, 0)
t= 8 S=(1, 0) A=→ R=-5.0 S'=(1, 1) x TRAMPA

Pasos totales : 9
Resultado final : TRAMPA (1, 1)
Recompensa acumulada : -5.8000
Retorno G_0 (γ=0.95): -3.9903
```

► Episodio 2/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

```
t= 0 S=(0, 0) A=↑ R=-0.1 S'=(0, 0)
t= 1 S=(0, 0) A=↑ R=-0.1 S'=(0, 0)
t= 2 S=(0, 0) A=↑ R=-0.1 S'=(0, 0)
t= 3 S=(0, 0) A=↓ R=-0.1 S'=(1, 0)

t= 4 S=(1, 0) A=↓ R=-0.1 S'=(2, 0)
t= 5 S=(2, 0) A=↑ R=-0.1 S'=(1, 0)
t= 6 S=(1, 0) A=↓ R=-0.1 S'=(2, 0)
t= 7 S=(2, 0) A=→ R=-0.1 S'=(2, 1)
t= 8 S=(2, 1) A=↓ R=-0.1 S'=(3, 1)
t= 9 S=(3, 1) A=→ R=-0.1 S'=(3, 2)
t=10 S=(3, 2) A=← R=-0.1 S'=(3, 1)
t=11 S=(3, 1) A=↑ R=-0.1 S'=(2, 1)
t=12 S=(2, 1) A=↓ R=-0.1 S'=(3, 1)
t=13 S=(3, 1) A=→ R=-0.1 S'=(3, 2)
t=14 S=(3, 2) A=← R=-0.1 S'=(3, 1)
t=15 S=(3, 1) A=← R=-0.1 S'=(3, 0)
t=16 S=(3, 0) A=↓ R=-0.1 S'=(3, 0)
t=17 S=(3, 0) A=↓ R=-0.1 S'=(3, 0)
t=18 S=(3, 0) A=← R=-0.1 S'=(3, 0)
t=19 S=(3, 0) A=↑ R=-0.1 S'=(2, 0)
```


t=20 S=(2, 0) A=↑ R=-0.1 S'=(1, 0)
t=21 S=(1, 0) A=→ R=-5.0 S'=(1, 1) x TRAMPA

Pasos totales : 22
Resultado final : TRAMPA (1, 1)
Recompensa acumulada : -7.1000
Retorno G_0 (γ=0.95): -3.0217

► Episodio 3/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

t= 0 S=(0, 0) A=↑ R=-0.1 S'=(0, 0)
t= 1 S=(0, 0) A=→ R=-0.1 S'=(0, 0)
t= 2 S=(0, 0) A=→ R=-0.1 S'=(0, 0)
t= 3 S=(0, 0) A=→ R=-0.1 S'=(0, 0)
t= 4 S=(0, 0) A=↑ R=-0.1 S'=(0, 0)
t= 5 S=(0, 0) A=→ R=-0.1 S'=(0, 1)
t= 6 S=(0, 1) A=↑ R=-0.1 S'=(0, 1)
t= 7 S=(0, 1) A=→ R=-0.1 S'=(0, 2)
t= 8 S=(0, 2) A=↑ R=-0.1 S'=(0, 2)
t= 9 S=(0, 2) A=→ R=-0.1 S'=(0, 1)

t=10 S=(0, 1) A=→ R=-0.1 S'=(0, 0)
t=11 S=(0, 0) A=↓ R=-0.1 S'=(1, 0)
t=12 S=(1, 0) A=↑ R=-0.1 S'=(0, 0)
t=13 S=(0, 0) A=↑ R=-0.1 S'=(0, 0)
t=14 S=(0, 0) A=↓ R=-0.1 S'=(1, 0)
t=15 S=(1, 0) A=→ R=-0.1 S'=(1, 0)
t=16 S=(1, 0) A=↑ R=-0.1 S'=(0, 0)
t=17 S=(0, 0) A=↓ R=-0.1 S'=(1, 0)
t=18 S=(1, 0) A=↑ R=-0.1 S'=(0, 0)
t=19 S=(0, 0) A=→ R=-0.1 S'=(0, 1)
t=20 S=(0, 1) A=→ R=-0.1 S'=(0, 0)
t=21 S=(0, 0) A=→ R=-0.1 S'=(0, 1)
t=22 S=(0, 1) A=→ R=-0.1 S'=(0, 0)
t=23 S=(0, 0) A=↓ R=-0.1 S'=(1, 0)
t=24 S=(1, 0) A=→ R=-0.1 S'=(1, 0)
t=25 S=(1, 0) A=→ R=-0.1 S'=(1, 0)
t=26 S=(1, 0) A=↓ R=-0.1 S'=(2, 0)
t=27 S=(2, 0) A=→ R=-0.1 S'=(2, 0)
t=28 S=(2, 0) A=↑ R=-0.1 S'=(1, 0)
t=29 S=(1, 0) A=↓ R=-0.1 S'=(2, 0)
t=30 S=(2, 0) A=↓ R=-0.1 S'=(3, 0)
t=31 S=(3, 0) A=↓ R=-0.1 S'=(3, 0)
t=32 S=(3, 0) A=→ R=-0.1 S'=(3, 1)
t=33 S=(3, 1) A=→ R=-0.1 S'=(3, 2)
t=34 S=(3, 2) A=→ R=-0.1 S'=(3, 1)
t=35 S=(3, 1) A=↓ R=-0.1 S'=(3, 1)
t=36 S=(3, 1) A=→ R=-0.1 S'=(3, 0)
t=37 S=(3, 0) A=↑ R=-0.1 S'=(2, 0)
t=38 S=(2, 0) A=↓ R=-0.1 S'=(3, 0)
t=39 S=(3, 0) A=↑ R=-0.1 S'=(2, 0)
t=40 S=(2, 0) A=→ R=-0.1 S'=(2, 0)
t=41 S=(2, 0) A=→ R=-0.1 S'=(2, 1)
t=42 S=(2, 1) A=→ R=-0.1 S'=(2, 0)
t=43 S=(2, 0) A=↑ R=-0.1 S'=(1, 0)
t=44 S=(1, 0) A=↓ R=-0.1 S'=(2, 0)
t=45 S=(2, 0) A=→ R=-0.1 S'=(2, 0)
t=46 S=(2, 0) A=↓ R=-0.1 S'=(3, 0)
t=47 S=(3, 0) A=→ R=-0.1 S'=(3, 1)
t=48 S=(3, 1) A=→ R=-0.1 S'=(3, 2)
t=49 S=(3, 2) A=→ R=+10.0 S'=(3, 3) ✓ META

Pasos totales : 50
Resultado final : META
Recompensa acumulada : +5.1000
Retorno G_0 (γ=0.95): -1.0281

► Episodio 4/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

t= 0 S=(0, 0) A=↓ R=-0.1 S'=(1, 0)
t= 1 S=(1, 0) A=→ R=-0.1 S'=(1, 0)
t= 2 S=(1, 0) A=↓ R=-0.1 S'=(2, 0)
t= 3 S=(2, 0) A=↓ R=-0.1 S'=(3, 0)
t= 4 S=(3, 0) A=→ R=-0.1 S'=(3, 0)
t= 5 S=(3, 0) A=→ R=-0.1 S'=(3, 1)
t= 6 S=(3, 1) A=→ R=-0.1 S'=(3, 2)
t= 7 S=(3, 2) A=→ R=-0.1 S'=(3, 1)
t= 8 S=(3, 1) A=↓ R=-0.1 S'=(3, 1)
t= 9 S=(3, 1) A=↓ R=-0.1 S'=(3, 1)
t=10 S=(3, 1) A=→ R=-0.1 S'=(3, 2)
t=11 S=(3, 2) A=↑ R=-0.1 S'=(2, 2)
t=12 S=(2, 2) A=↑ R=-0.1 S'=(1, 2)
t=13 S=(1, 2) A=↑ R=-0.1 S'=(0, 2)
t=14 S=(0, 2) A=↓ R=-0.1 S'=(1, 2)
t=15 S=(1, 2) A=↓ R=-0.1 S'=(2, 2)
t=16 S=(2, 2) A=→ R=-5.0 S'=(2, 3) x TRAMPA

Pasos totales : 17
Resultado final : TRAMPA (2, 3)
Recompensa acumulada : -6.6000
Retorno G_0 (γ=0.95): -3.3204

► Episodio 5/5

INICIO DEL EPISODIO | Estado inicial: (0, 0)

t= 0 S=(0, 0) A=↑ R=-0.1 S'=(0, 0)
t= 1 S=(0, 0) A=→ R=-0.1 S'=(0, 1)
t= 2 S=(0, 1) A=→ R=-0.1 S'=(0, 2)
t= 3 S=(0, 2) A=→ R=-0.1 S'=(0, 3)

```
t= 4 S=(0, 3) A=- R=-0.1 S'=(0, 2)
t= 5 S=(0, 2) A=+ R=-0.1 S'=(0, 2)
t= 6 S=(0, 2) A=+ R=-0.1 S'=(0, 2)
t= 7 S=(0, 2) A=- R=-0.1 S'=(0, 1)
t= 8 S=(0, 1) A=- R=-0.1 S'=(0, 0)
t= 9 S=(0, 0) A=+ R=-0.1 S'=(0, 0)
t=10 S=(0, 0) A=- R=-0.1 S'=(0, 0)
t=11 S=(0, 0) A=+ R=-0.1 S'=(0, 1)
t=12 S=(0, 1) A=- R=-5.0 S'=(1, 1) x TRAMPA
```

```
Pasos totales      : 13
Resultado final    : TRAMPA (1, 1)
Recompensa acumulada : -6.2000
Retorno G_0 (γ=0.95): -3.6211
```

RESUMEN DEL EXPERIMENTO

Retornos G_0 por episodio:

```
Episodio 1: -3.9903
Episodio 2: -3.0217
Episodio 3: -1.0281
Episodio 4: -3.3204
Episodio 5: -3.6211
```

Estadísticas de los retornos:

```
Retorno promedio G_0 : -2.9963
Desviación estándar  : 1.0352
Retorno máximo       : -1.0281
Retorno mínimo       : -3.9903
Promedio de pasos    : 22.20
```

Distribución de resultados:

```
TRAMPA (1, 1): 3 episodio(s)
META: 1 episodio(s)
TRAMPA (2, 3): 1 episodio(s)
```

COMPARACIÓN FINAL

PREGUNTA 2: NUEVA TRAMPA EN (0, 3)

```
Promedio original      : -13.5606
Promedio con nueva trampa: -12.6096
Desviación original    : 2.5684
Desviación nueva trampa : 3.6584
```

Retornos originales:

```
[-10.0487, -14.8916, -17.5702, -13.3981, -11.8946]
```

Retornos con la trampa (0, 3):

```
[-10.0487, -14.8916, -17.5702, -13.3981, -7.1394]
```

Resultados terminales originales:

```
{'TRAMPA (1, 1)': 3, 'META': 1, 'TRAMPA (2, 3)': 1}
```

Resultados terminales con nueva trampa:

```
{'TRAMPA (1, 1)': 2, 'META': 1, 'TRAMPA (2, 3)': 1, 'TRAMPA (0, 3)': 1}
```

Veces que el agente cayó en la nueva trampa: 1

PREGUNTA 3: RECOMPENSA POR PASO DE -0.1

```
Promedio con paso -1.0 : -13.5606
Promedio con paso -0.1 : -2.9963
```

Retornos con recompensa por paso de -1.0:

```
[-10.0487, -14.8916, -17.5702, -13.3981, -11.8946]
```

Retornos con recompensa por paso de -0.1:

```
[-3.9903, -3.0217, -1.0281, -3.3204, -3.6211]
```

Promedio de pasos con recompensa -1.0: 22.20

Promedio de pasos con recompensa -0.1: 22.20

¿Las acciones y trayectorias fueron iguales al cambiar la recompensa?
True

Nota: el agente utiliza una política aleatoria y no aprende. Por eso, cambiar la recompensa modifica los retornos, pero no cambia la forma en que selecciona sus acciones.

2. Nueva trampa en la posición (0, 3)

Output del escenario original

Retornos originales:

```
[-10.0487, -14.8916, -17.5702, -13.3981, -11.8946]
```

Retorno promedio G_0 : -13.5606

Desviación estándar : 2.5684

Distribución de resultados:

```
TRAMPA (1, 1): 3 episodios
```

```
META: 1 episodio
```

```
TRAMPA (2, 3): 1 episodio
```

Output con la nueva trampa en (0, 3)

```
Retornos con la trampa (0, 3):  
[-10.0487, -14.8916, -17.5702, -13.3981, -7.1394]  
  
Retorno promedio G_0 : -12.6096  
Desviación estándar : 3.6584  
  
Distribución de resultados:  
TRAMPA (1, 1): 2 episodios  
META: 1 episodio  
TRAMPA (2, 3): 1 episodio  
TRAMPA (0, 3): 1 episodio  
  
Veces que el agente cayó en la nueva trampa: 1
```

Análisis

Agregar la trampa en (0, 3) hizo que uno de los episodios terminara en solo cuatro pasos, con un retorno de -7.1394, en lugar de continuar hasta caer posteriormente en otra trampa. Por esta razón, el retorno promedio fue ligeramente menos negativo, pero la desviación estándar aumentó de 2.5684 a 3.6584, lo que indica una distribución de retornos más variable. Aunque una nueva trampa normalmente representa un mayor riesgo, en este caso terminar temprano evitó acumular más recompensas de -1 por paso.

3. Recompensa por paso modificada de -1 a -0.1

Output con recompensa por paso de -1

```
Retornos con recompensa por paso de -1.0:  
[-10.0487, -14.8916, -17.5702, -13.3981, -11.8946]  
  
Retorno promedio G_0 : -13.5606  
Promedio de pasos : 22.20
```

Output con recompensa por paso de -0.1

```
Retornos con recompensa por paso de -0.1:  
[-3.9903, -3.0217, -1.0281, -3.3204, -3.6211]  
  
Retorno promedio G_0 : -2.9963  
Promedio de pasos : 22.20  
  
¿Las acciones y trayectorias fueron iguales?  
True
```

Análisis

Al cambiar la recompensa por paso de -1 a -0.1, el retorno promedio mejoró considerablemente, pasando de -13.5606 a -2.9963, porque cada movimiento produjo una penalización mucho menor. Sin embargo, el comportamiento del agente no cambió: el promedio de pasos continuó siendo 22.20 y las trayectorias fueron iguales, ya que el agente sigue una política aleatoria y no utiliza las recompensas para aprender o modificar sus acciones.